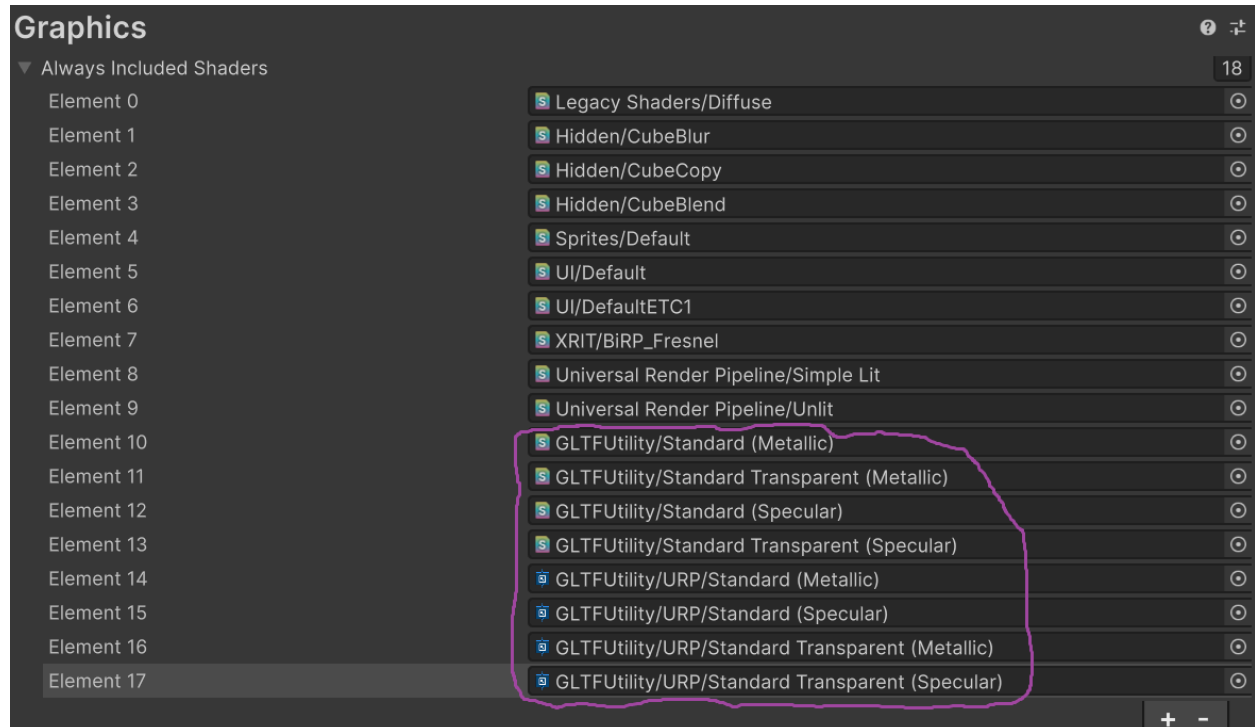


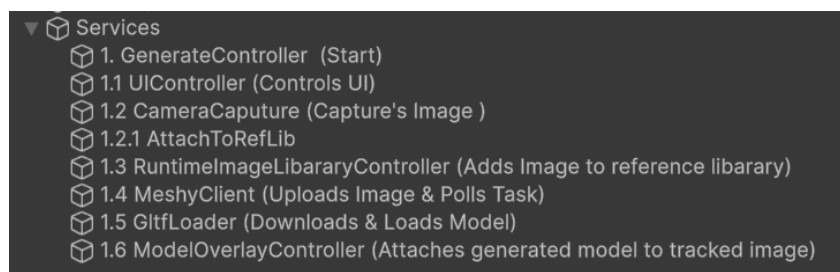
Eureka!: 20/11/2025

Continuing from the last debugging session, we figured out that the library glTFast wasn't able to format the textures from the Meshy API, and I was in the process of ripping out the current skeleton, glTFast, and replacing it with GLTFUtility. When this change was made, I also included in the graphics settings every shader involving the library, as shown in the screenshot below.



After tinkering with the script settings within "GltfLoader.cs" we finally got the textures to work as intended, now the whole project is complete, where the steps are as follows.

1. Take a picture with your mobile device (Android)
2. That picture gets processed with the Reference Library in a higher resolution and gets sent forward to the Meshy API. "**CameraCapture.cs**"
3. Meshy processes the request and polls the progression of the generation to our UI. "**MeshyClient.cs**"
4. GltfLoader.cs then handles the model during runtime and unpacks the textures using the library GLTFUtility. "**GltfLoader.cs**"
5. Lastly after processing that model is binded to the picture taken during step 1. And displayed fully with textures. "**ModelOverlayController.cs**"



Conclusion:

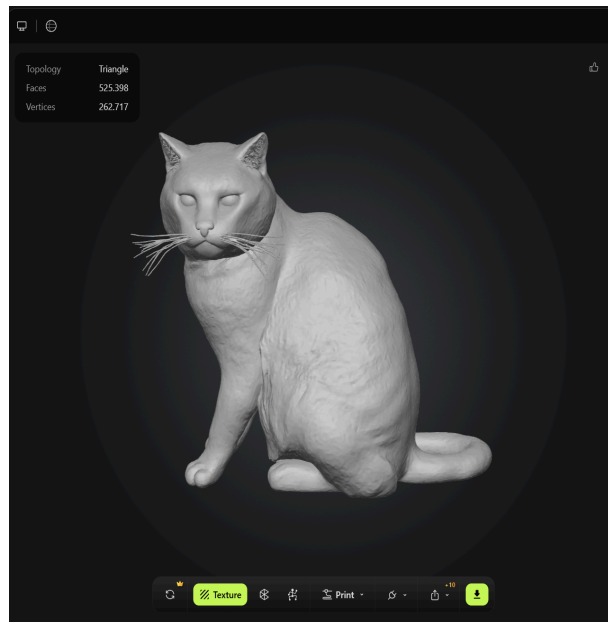
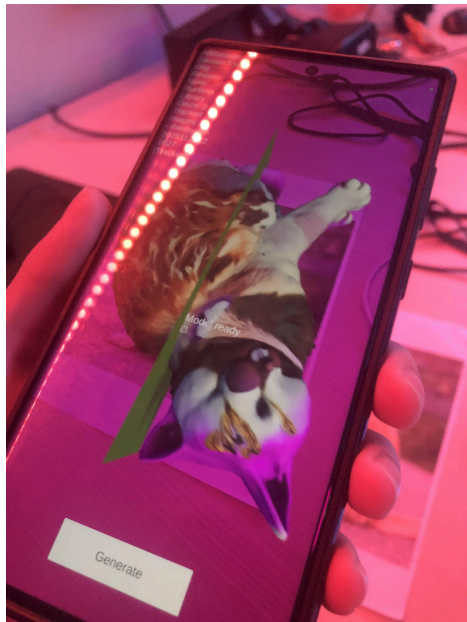
Looking back now, glTFast could perhaps have worked if I had included its shaders in the graphics settings, but I can't recall ever seeing existing shaders for that library. But that's beside the point — we have now completed this project to a level that can be presented.

In addition, our requirements changed from a simple AR-tracking project to what it eventually became, and the work finally paid off. We went from a static, predefined AR image tracker to a dynamic, runtime AR application. I would say it's one of the very first attempts within this field, and the risk paid off.

Future:

If we had more time, I would've experimented more with different pictures. During testing it seemed like pictures without a background/transparent backgrounds had a higher chance for Meshy to generate a model based on the paper itself rather than the contents of the picture. However, with our orange cat (with the full background), Meshy could easily generate the cat in decent quality.

If we had more time, we would've perhaps looked more closely at the Meshy documentation to see if there was a difference between using the API and generating the cat on the website. I tested this, and there seems to be a big difference.



Black and white was generated on the website.

Mobile screenshot was generated via the Meshy API.